SECTION 03 · ANOMALY DETECTION

Anomaly Detection for Credit Card Fraud

Score 284,807 transactions without using the fraud labels, then use the labels only to grade the ranking.

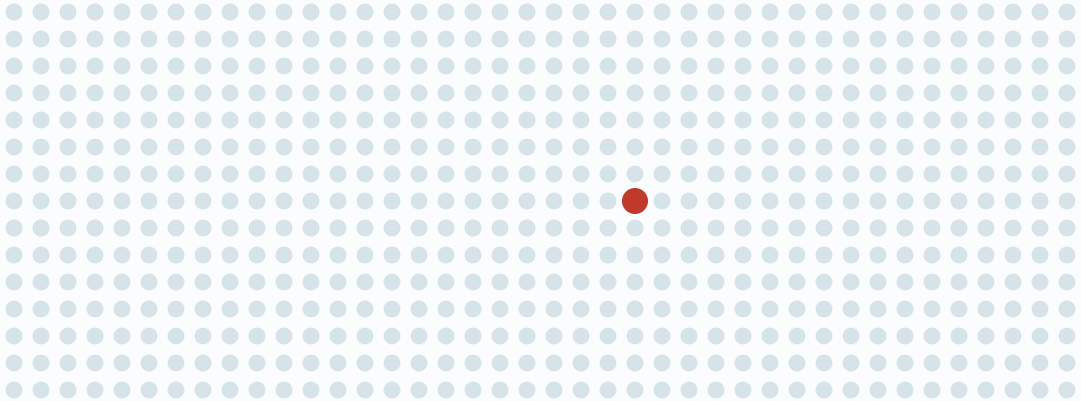
Reconstruction Error | Precision-Recall | PCA | Isolation Forest | LOF | ECOD

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

Why Accuracy Fails at 0.17% Fraud



600 transactions: about one is fraud

fraud is 0.17% of the rows in this dataset

A model that never says "fraud"
is right 99.8% of the time
and catches nothing

Accuracy rewards ignoring the rare class.

Fraud teams care about two other questions:

- **of the alerts we raise, how many are fraud?**
- **of all the fraud, how much do we catch?**

Unsupervised Anomaly Detection

transactions
no labels used

→

unsupervised model
learns what is normal

→

a score per row
higher = stranger

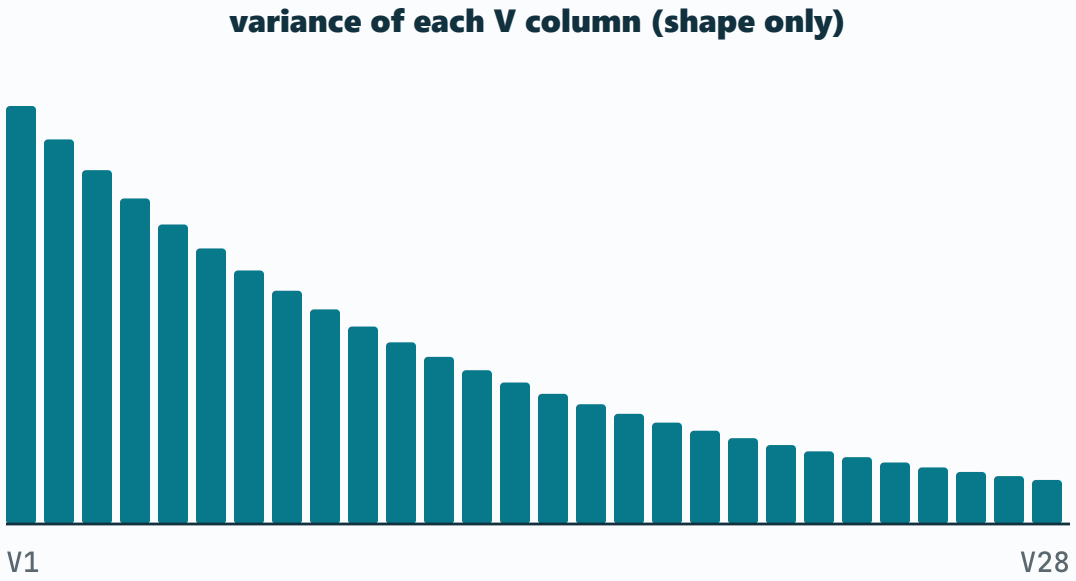
→



the top goes to a human

The fraud labels stay hidden.
We use them only to grade the ranking.

The Features Are Already PCA Scores

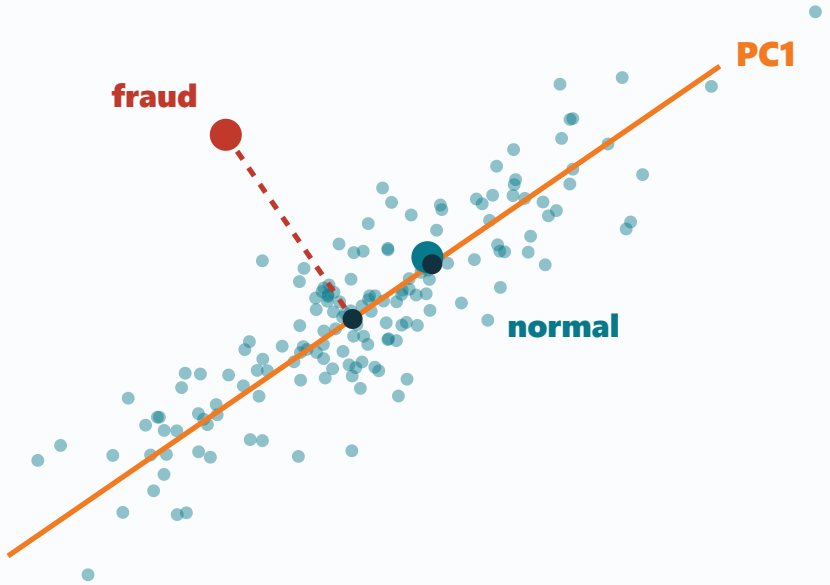


falls from V1 to V28, like PCA components

The bank already ran PCA to hide the original fields

`V1 ... V28`	PCA scores, uncorrelated, keep as they are
`Amount`	long right tail: log1p pulls it in
`Time`	seconds since the first row: dropped
`Class`	the label: kept aside to grade

Reconstruction Error as a Fraud Score



dashed line = what PCA cannot rebuild

1. Fit PCA with fewer components than features

2. Project each row down and back up

3. Score = squared distance between the row and its rebuilt copy

Components follow the bulk of the data,
so normal rows rebuild well and rare,
unusual rows are left far off the plane

How Many Components to Keep

all components



every row rebuilt exactly

error is zero for everyone: no signal

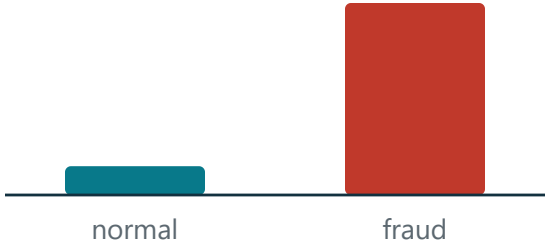
very few



normal rows lose detail too

fraud hides among large errors

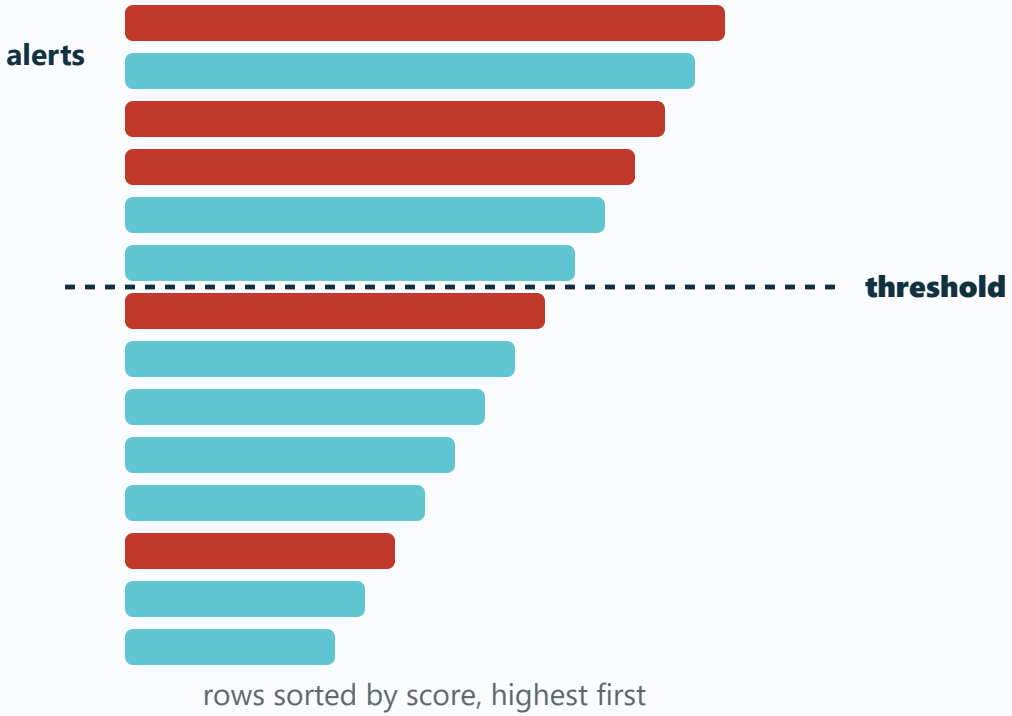
a middle number



normal rows rebuilt, fraud not

error separates the two

Precision and Recall on a Ranked List



Precision = frauds above the line / rows above it

"How many of my alerts are real?"

here: 3 of 6

Recall = frauds above the line / all frauds

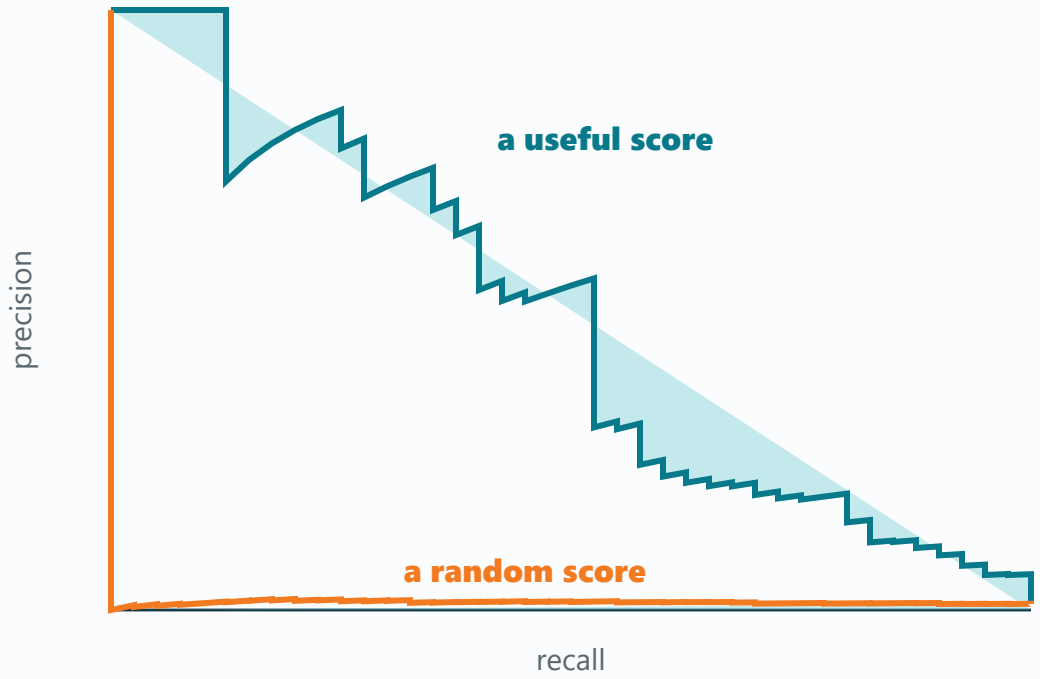
"How much fraud do I catch?"

here: 3 of 5

Lower the line: recall goes up, precision usually falls.

Raise it: fewer, cleaner alerts, more fraud missed.

The Precision-Recall Curve and AP



Slide the threshold from top to bottom and plot precision against recall

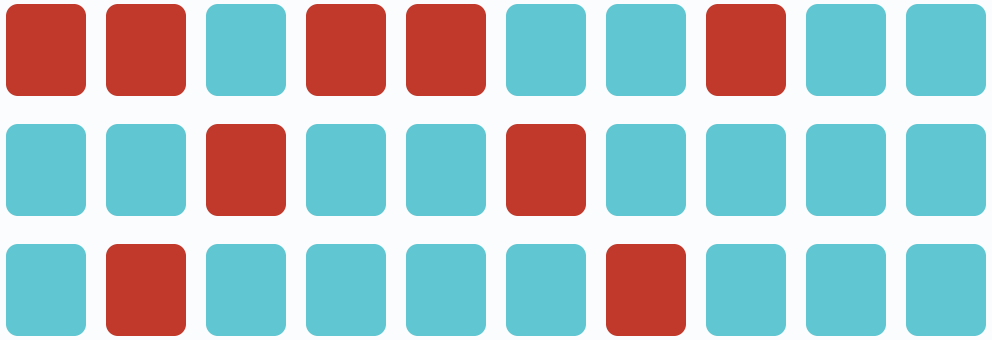
Average precision (AP) = the shaded area, one number for the whole ranking

Random scoring sits at the fraud rate, close to zero. Perfect scoring is 1.

ROC AUC flatters rare classes
it can look high while most alerts are false; precision and recall do not hide that

Top-N Precision as a Review Budget

the analyst's queue: the N highest scores



red = fraud found, pale = false alarm

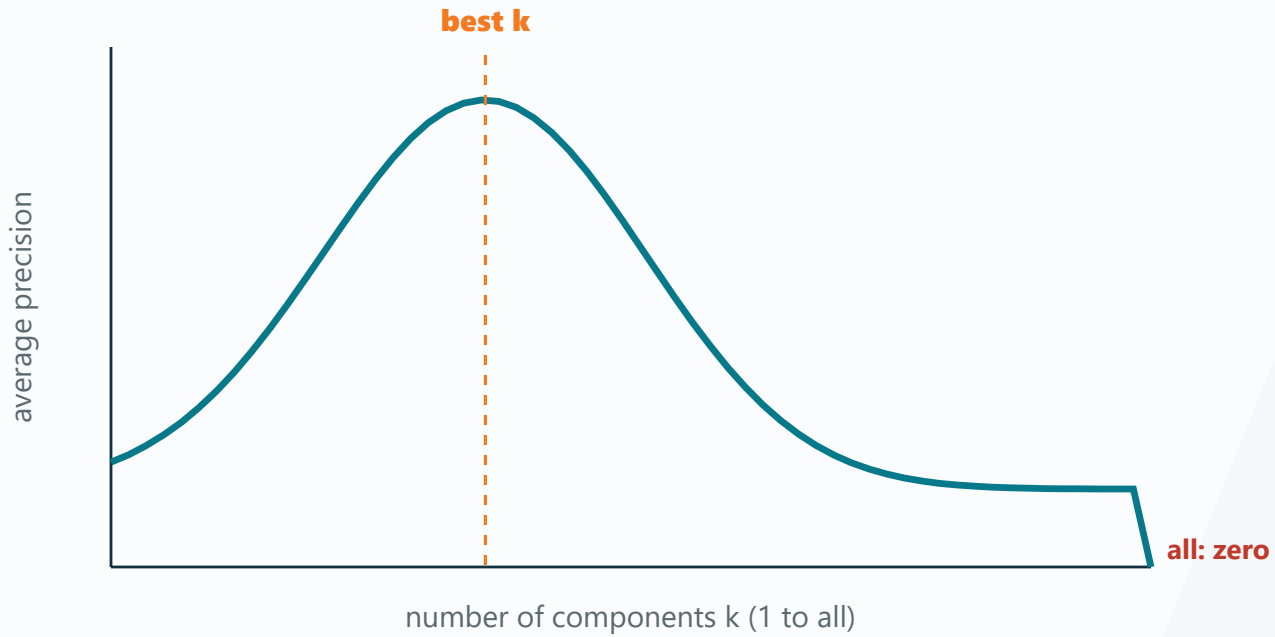
A team can only check so many transactions a day

Precision at N: share of the top N that are fraud

Recall at N: share of all fraud inside the top N

Set N to the number of frauds
a perfect score would fill the queue with fraud

Choosing the Number of Components



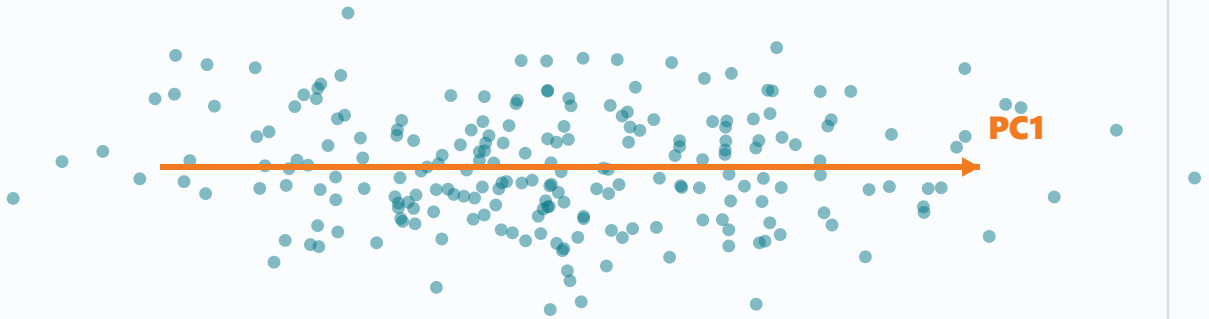
No labels, no right answer for k

Here we can peek at held-back labels
on the training rows to pick k,
then judge it once on the test rows.

With no labels at all
keep enough components for most of the variance, and check the top alerts by hand

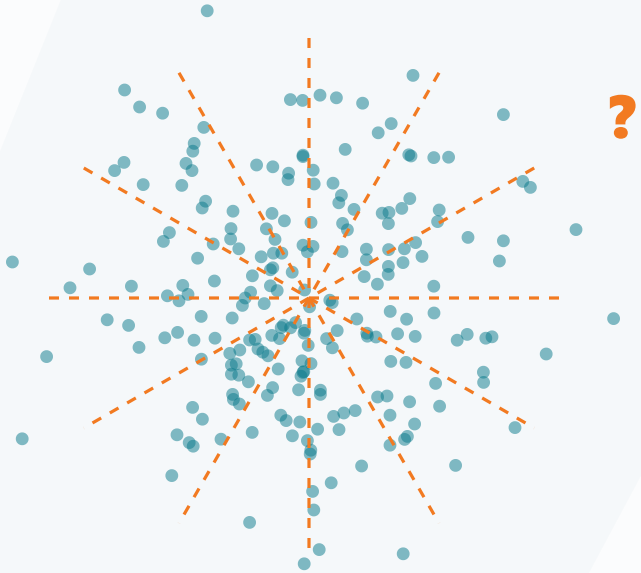
Why Standardising Hurts Here

as given: V columns



one clear long direction

after StandardScaler



a round cloud, no direction wins

The V columns are uncorrelated with falling variance. Scaling erases that order, so PCA picks almost arbitrary directions.

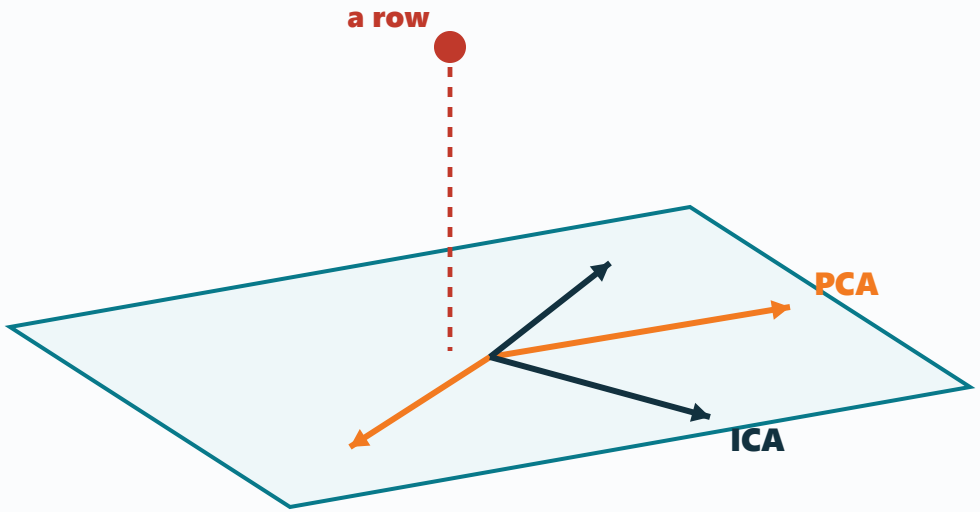
Other Ways to Rebuild a Row

Sparse PCA	components that use only a few features
ICA	independent directions, rotated inside the PCA plane
Random projection	a random plane; cheap, keeps distances, not the data's shape
Dictionary learning	each row built from a few learned atoms
Kernel PCA	a curved surface instead of a flat plane

Same recipe for all: fit, transform, inverse_transform, squared error.

Why ICA Scores Like PCA

Both pick k directions inside the same plane



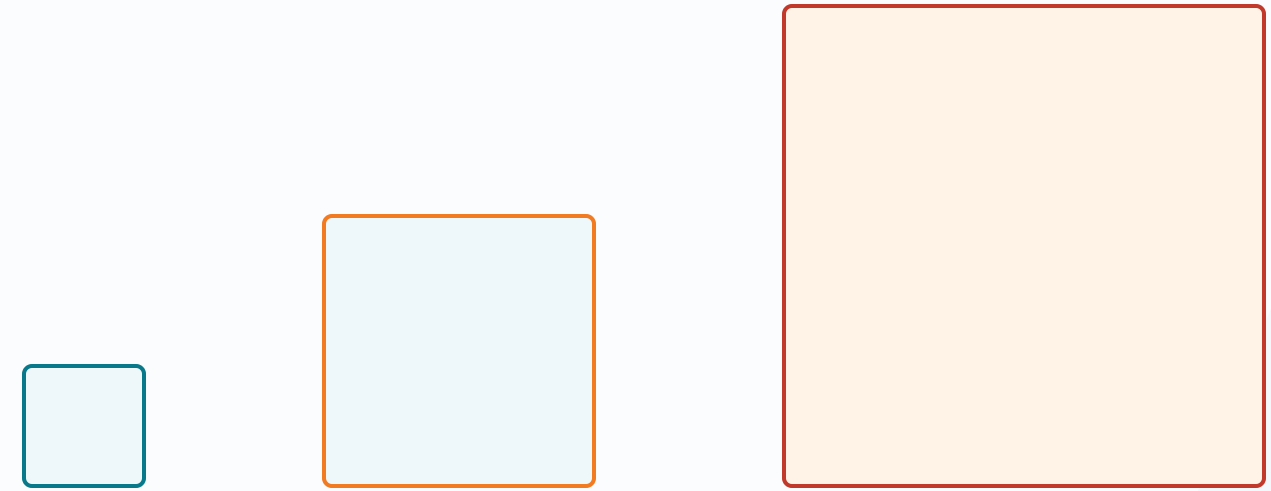
FastICA first whitens the data with PCA, then rotates the axes inside that space

Different axes, same plane:
every row lands on the same rebuilt point

Same plane, same error, same score
ICA helps to separate signals, not here

Kernel PCA Needs a Sample

Kernel matrix: one value per pair of rows



2,000 rows

10,000 rows

190,000 rows

rows squared: 190,000 rows would need about 270 GB

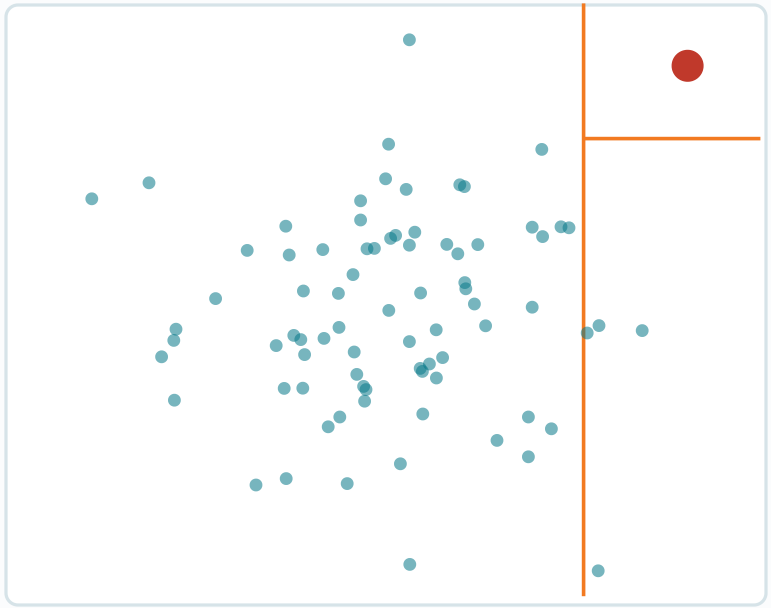
Kernel PCA compares every row with every training row

Fit on a small sample
then transform the rest; each new row is compared with the sample only

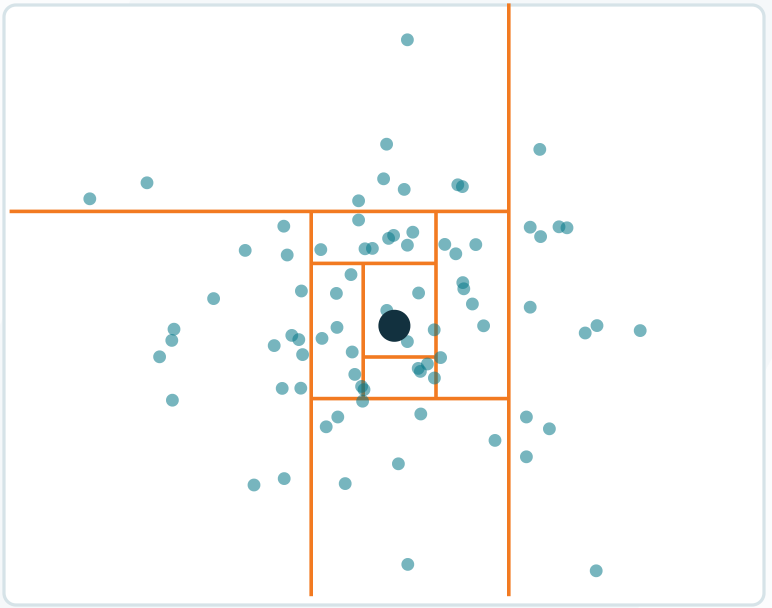
`fit_inverse_transform=True` learns the way back, so it can be scored like PCA

How Isolation Forest Isolates a Point

outlier: isolated in 2 splits

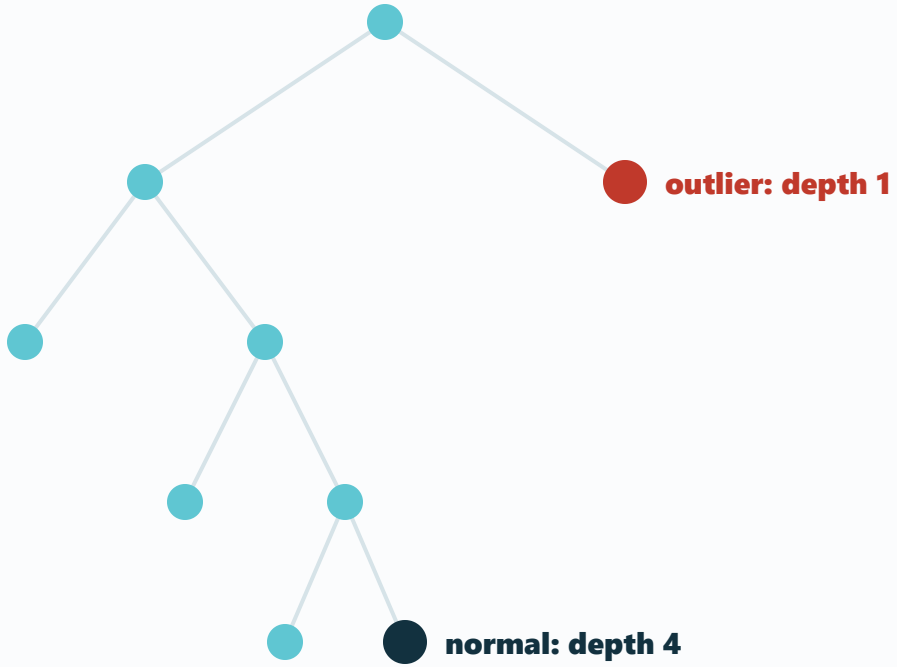


normal point: many splits



Random feature, random cut, repeat. Outliers end up alone after very few cuts.

Path Length Becomes the Score

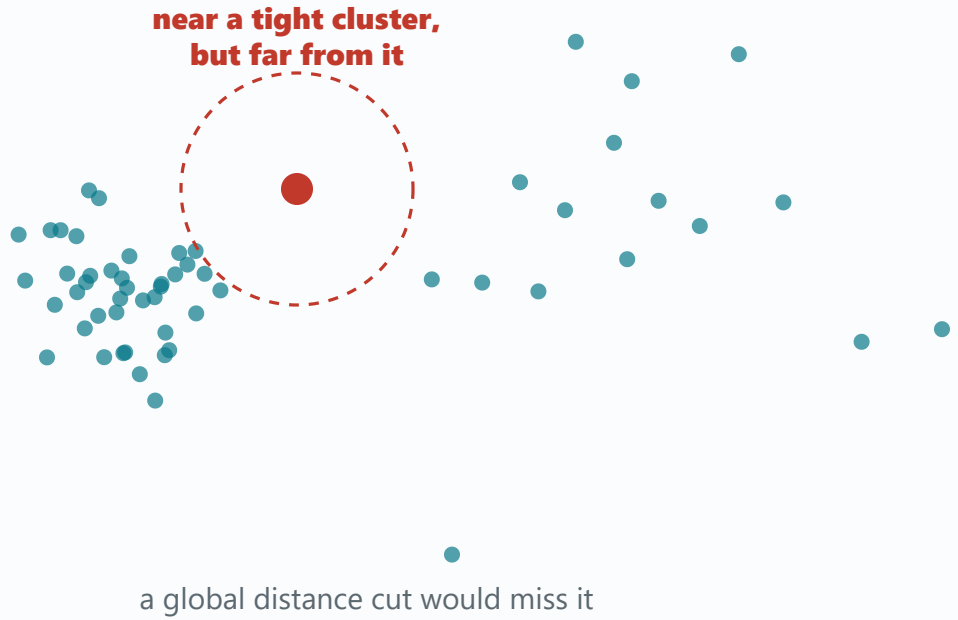


Each tree splits at random until every row sits alone in a leaf

Score = average depth over many trees, short paths mean anomalies

score_samples is high for normal rows
so flip the sign to rank fraud first;
fast, no distances, handles many rows

Local Outlier Factor



LOF compares a point's density with the density of its neighbours

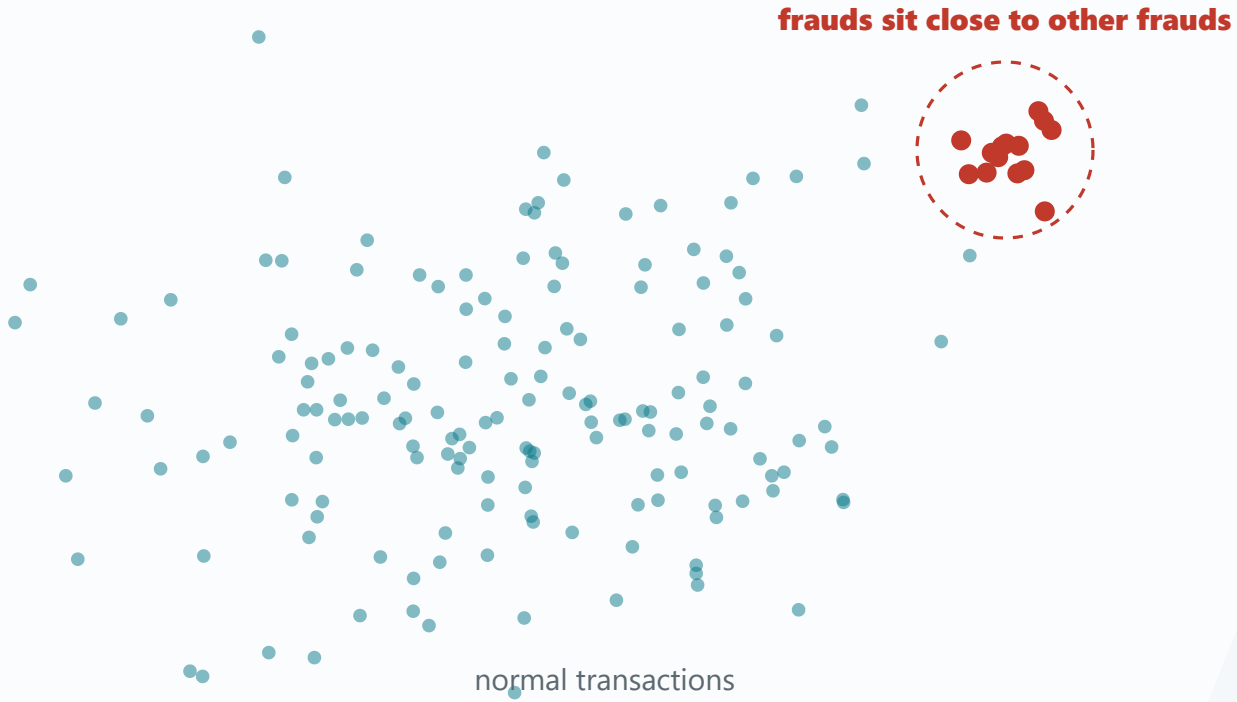
about 1: as crowded as its neighbours

well above 1: lonelier than its neighbours

Local, not global

it finds outliers next to dense groups
that one distance cut would miss

Why LOF Misses Grouped Fraud



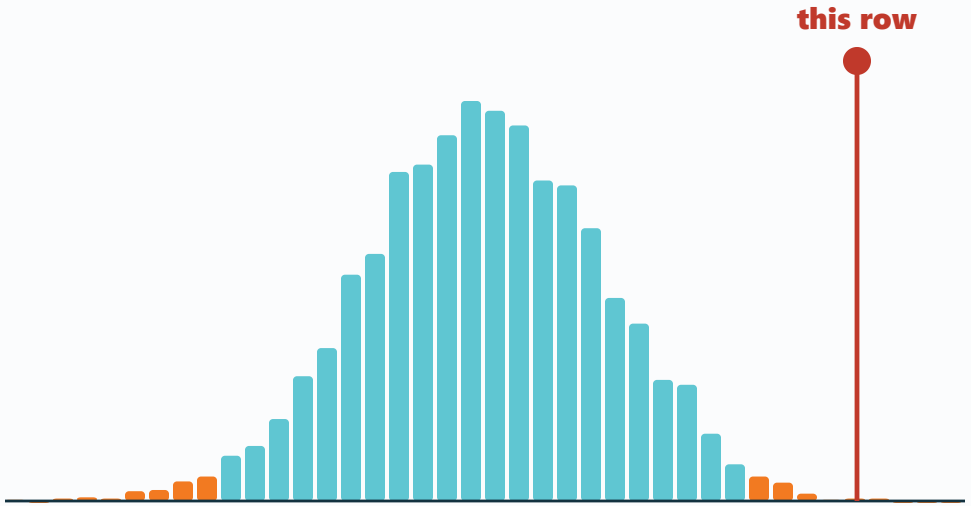
A fraud's nearest neighbours are mostly other frauds

Same pattern repeated, even exact duplicates, so each fraud looks as crowded as its neighbours

LOF scores them as normal

a method's assumption must match the data; here global methods win

ECOD Scores Each Feature's Tail



one feature, all training rows

orange = the tails

For every feature: how far into its own tail does this value sit?

1. **Empirical distribution per column**
2. **Tail probability of the row's value**
3. **Add up minus log of those probabilities**

No settings, no training loop

a strong first baseline from PyOD;

it misses fraud that only shows in combinations

One Honest Check on the Test Set

training rows (two thirds)

fit the detector, choose k
compare methods

test rows

score once at the very end
no more changes after

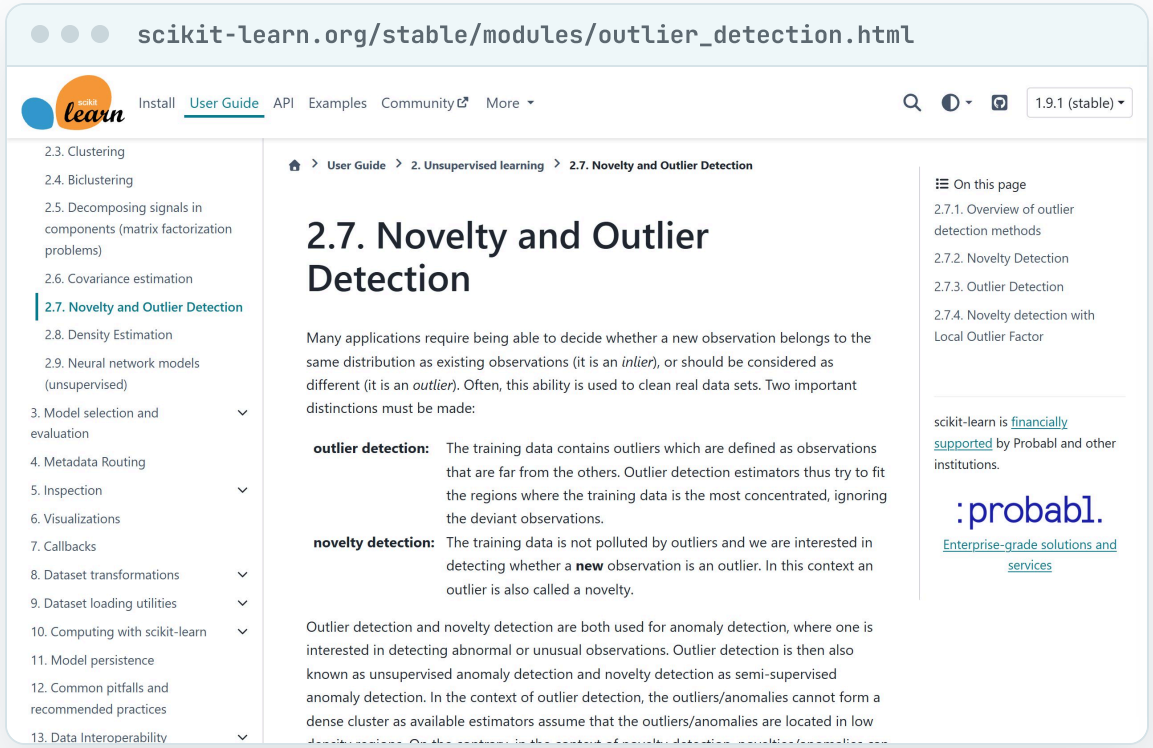
`stratify=y`

same fraud rate
in both parts

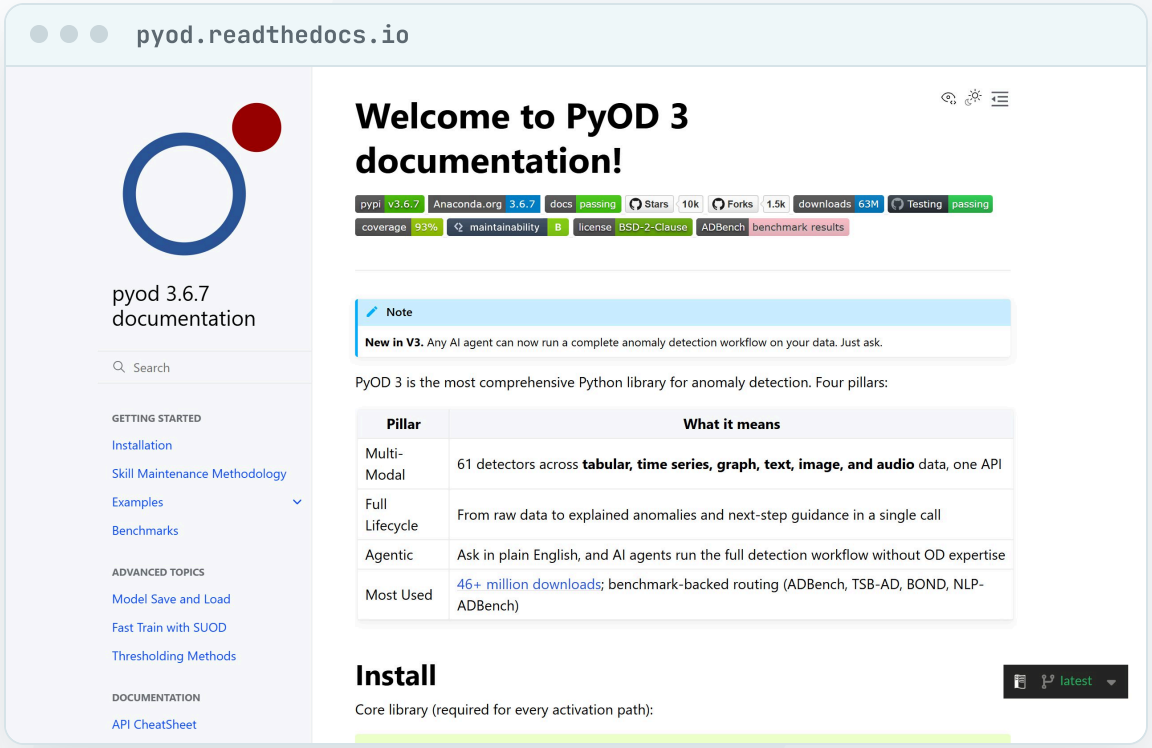
Choosing k on the test rows would make the test score look better than it is

The test score is the honest estimate for new transactions

scikit-learn and PyOD Docs



scikit-learn: Isolation Forest, LOF and more



PyOD: dozens of detectors behind one API, ECOD among them

PyOD detectors share fit, decision_scores_ and decision_function, like scikit-learn.

Choosing an Anomaly Detector

DETECTOR	LOOKS FOR	COST	WATCH OUT FOR
PCA reconstruction	rows off the main plane	fast	needs k; scaling choice matters
Sparse PCA, dictionary	same, with simpler parts	slower fit	more settings
ICA	same plane as PCA	fast	adds nothing for scoring
Random projection	a random plane	fastest	ignores the data's shape
Kernel PCA	rows off a curved surface	rows squared	fit on a sample
Isolation Forest	rows easy to cut off	fast	axis-aligned cuts
LOF	lonelier than neighbours	neighbour search	anomalies that come in groups
ECOD	extreme single features	fast, no settings	odd combinations of normal values

SECTION 03 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie